

Assignment 02

CSE 241

May 22, 2024

You are to finish a C++ symbolic calculation project. The parsing code has already been written for you. You must complete the program by creating a class hierarchy to represent an expression tree, and implement certain operations, such as printing an expression or calculating the derivative of an expression. You must also write a `main` function to read in input and produce output.

Description

Your program will prompt the user for a mathematical expression. The parse function that has been provided for you will construct an expression tree. You will then ask the user for an integer value of x to evaluate the expression at, and print out the result. Then, symbolically, you will calculate the derivative of the expression, print out the derivative, and print out the derivative evaluated at the given value of x .

Mathematical expressions

As stated previously, the parsing code will produce an expression tree. The expression tree will be made of 5 different types. Note that, in order for the parse code to work correctly, you must name your classes *exactly* the way they are stated here, and you must have constructors in your classes that match exactly as given here.

There are two types of leaf nodes:

`int_literal` represents a long integer value, such as 3 or -5. This class must have a unary constructor with a `long` parameter.

`monomial` represents the variable x raised to the power of some long integer exponent. This class must also have a unary constructor with a `long` parameter. Note that only the variable x is considered in this program for mathematical monomials. E.g., the monomial x^7 would be constructed as `new monomial(7)`.¹

¹ Monomials in this expression tree do *not* have coefficients.

There are three types of internal nodes of the expression tree:

`addition` represents two mathematical expressions being added together. It must have a binary constructor, with two parameters of type `expr*`.

`multiplication` represents two mathematical expressions being multiplied together. It also must have a binary constructor, with two parameters of type `expr*`.

`division` represents two mathematical expressions being divided. It also must have a binary constructor, with two parameters of

type `expr*`. The first parameter is the numerator² and the second parameter is the denominator³ of the division.

² top

³ bottom

As an example, $\frac{3x^2}{x-1}$ would be constructed as the following snippet of code:

```
new division(new multiplication(new int_literal(3), new monomial(2)), new
    addition(new monomial(0), new int_literal(-1)))
```

It is your code's responsibility to manage dynamic memory after construction, such that there are no memory leaks or dangling pointers.

Output

When outputting the function evaluated at a particular value of x , you will produce the value as a double, and print it using the default output style for doubles.

When outputting a symbolic expression, to get a maximum score of 90%, your output must merely be correct⁴. To get 100%, you must implement some simplifications or optimizations in either your expression tree or in the printing code such that certain things be eliminated, such as:

⁴ as checked by a human TA

- Superfluous multiplications by 1 or additions by 0 are simplified. E.g., $1 * x + 0$ should be written as merely x .
- Superfluous parentheses are simplified. E.g., $((2) * (x2)) + (3)$ should be written as merely $2 * x2 + 3$.
- Small exponents on monomials should be simplified. E.g., $x0$ should be 1, and $x1$ should be merely x .
- Integer constants should be combined together. E.g., $3 * 2 * x$ should be merely $6 * x$.

You will get some marks for each of these, individually, as specified in the rubric. ⁵ Some of these are more difficult than others. Focus on getting it working before attempting these simplifications!

⁵ Doing all of them (successfully) may get you more than 100%!

Derivatives

I know you learned this in introductory calculus, but I will give you the rules here for easy reference⁶.

- If $f(x) = n$ for some $n \in \mathbb{Z}$, then $f'(x) = 0$.
- If $f(x) = x^k$ for some $k \in \mathbb{Z}^+$, then $f'(x) = kx^{k-1}$.
- If $f(x) = g(x) + h(x)$, then $f'(x) = g'(x) + h'(x)$.
- If $f(x) = g(x)h(x)$, then $f'(x) = g'(x)h(x) + g(x)h'(x)$.
- If $f(x) = \frac{g(x)}{h(x)}$, then $f'(x) = \frac{g'(x)h(x) - g(x)h'(x)}{(h(x))^2}$.

⁶ For ease of typing, we use Euler's and Lagrange's "prime" notation everywhere for denoting derivatives. If you are used to Leibniz's notation, $f'(x)$ is simply $\frac{d}{dx}f(x)$.

Sample usages

Enter expression: 4

$f(x) = 4$

Enter x to evaluate at: 25

$f(25) = 4$

$f'(x) = 0$

$f'(25) = 0$

Enter expression: x

$f(x) = x$

Enter x to evaluate at: 25

$f(25) = 25$

$f'(x) = 1$

$f'(25) = 1$

Enter expression: $x^3 + x$

$f(x) = x^3 + x$

Enter x to evaluate at: 10

$f(10) = 1010$

$f'(x) = 3 * x^2 + 1$

$f'(10) = 301$

Enter expression: $(x^2 + 1) * (3x * (x + 1))$

$f(x) = 3 * x * (x + 1) * (x^2 + 1)$

Enter x to evaluate at: 2

$f(2) = 90$

$f'(x) = (3 * (x + 1) + 3 * x) * (x^2 + 1) + 3 * x * (x + 1) * 2 * x$

$f'(2) = 147$

Enter expression: $1 / x$

$f(x) = 1 / x$

Enter x to evaluate at: 2

$f(2) = 0.5$

$f'(x) = -1 / (x * x)$

$f'(2) = -0.25$

*Requirements**Restrictions*

You must *not* have any memory leaks or memory errors. Your submitted Makefile must compile with the address sanitizer enabled.

You may not use any global variables.

Submitting

You must submit a .zip file. The .zip file should contain:

- `parse.h`⁷
- `parse.cc`⁸
- `expr.h`
- `expr.cc`
- `main.cc`
- `Makefile`⁹

⁷ unchanged

⁸ unchanged

⁹ You must compile with the address sanitizer, and `make` must compile and link an executable called `main`

Nothing else should be in the .zip file.

No commenting or documentation is required with the submission. If you have received help from a friend, online, or used AI assistance, please add a comment to your submission saying so so that you cannot be processed for plagiarism.